

UNIVERSIDAD AUTÓNOMA “JUAN MISAEI SARACHO”

FACULTAD DE CIENCIAS Y TECNOLOGÍA · DIRECCIÓN DE POSGRADO

DIPLOMADO EN DESARROLLO WEB Y APLICACIONES MÓVILES · 2026

PERFIL DE PROYECTO — TRABAJO FINAL

Módulo 4 · Entregable E1 · Semana 1

Campo	Dato
Postulante	Jorge Luis Gutierrez Rivera
Correo electrónico	e73700@uajms.edu.bo
Título provisional del trabajo	Max Pizzapp: sistema web de gestión de pedidos con sincronización en tiempo real para la pizzería Max Pizzas
Línea de investigación	Desarrollo de aplicaciones web
Tipo de solución	web
Organización o ámbito beneficiario	Pizzería Max Pizzas — local central de la zona Villa Fátima, Tarija
Grupo de tutoría	Grupo 1 (lunes, 20:00–21:00)
Decisión de la tutoría T1	Aprobado con recorte
Alcance acordado en T1	El planteamiento inicial abarcaba la operación completa de los tres locales de Max Pizzas —central, Villa Avaroa y Tabladita—, con cuatro roles, control de inventario, cobro, cierre de caja y reportes consolidados. En la tutoría se acordó recortarlo al tramo de mayor valor : el flujo de un pedido entre recepción y cocina , con seguimiento de su estado en tiempo real , sobre un único local piloto (central, Villa Fátima) y dos roles diferenciados . Quedan fuera de alcance la operación multi-sede, el inventario, el cobro y el cierre de caja, el despacho como rol independiente y el rol administrador; todo ello se declara en la Tabla 9 y se recoge como trabajo futuro en el apartado 3.2.
Enlace al repositorio	https://github.com/jorgeluisgutierrezrivera/max-pizzapp
Enlace al tablero de tareas	https://github.com/users/jorgeluisgutierrezrivera/projects/2
Fecha de entrega	19 de septiembre de 2026

Tabla 1. Datos generales del perfil de proyecto.

CAPÍTULO 1. INTRODUCCIÓN

1.1 Antecedentes

La gestión de pedidos en el rubro gastronómico se ha ido informatizando en torno a tres piezas hoy de uso corriente: el punto de venta, donde se registra la comanda; la pantalla de cocina —*kitchen display system*, en adelante KDS—, que sustituye la comanda impresa mostrando los pedidos en el área de preparación; y el registro del estado de cada pedido a lo largo de su ciclo de vida. Vistas en conjunto, las tres atienden un mismo problema de fondo: dos puestos de trabajo —quien atiende y quien prepara— comparten un mismo objeto, el pedido, y necesitan verlo en el mismo estado al mismo tiempo. Cuando no existe soporte informático, esa coordinación se sostiene en la memoria de las personas y en lo que alcanzan a decirse mientras trabajan.

La forma habitual de construir este tipo de aplicaciones es la arquitectura cliente-servidor por capas: una interfaz que presenta y recoge datos, un servidor de aplicación que concentra las reglas de negocio y la autorización, y una base de datos que conserva el estado. La comunicación entre la interfaz y el servidor se organiza como una interfaz de programación de aplicaciones de estilo REST, cuyas restricciones —interfaz uniforme, ausencia de estado de sesión en el servidor y recursos identificados por dirección— fueron formuladas por Fielding (2000). Sobre una base de datos relacional, el modelo de datos aporta además trazabilidad: qué se pidió, quién modificó su estado y en qué momento, información que el registro en papel no conserva.

La expresión “en tiempo real” merece precisión, porque ahí se juega la diferencia con el proceso manual. El protocolo HTTP responde a un esquema de petición y respuesta: el servidor contesta cuando se le pregunta. Para reflejar en una pantalla un cambio ocurrido en otro puesto de trabajo existen dos caminos. El primero es la consulta periódica o *polling*: el cliente pregunta

cada cierto intervalo, lo que introduce una demora igual a ese intervalo y genera tráfico incluso cuando no hay novedades. El segundo es un canal bidireccional persistente sobre una única conexión, especificado como protocolo WebSocket en el RFC 6455 (Fette y Melnikov, 2011), que permite al servidor enviar el cambio en cuanto se produce. La diferencia práctica es la que separa *actualizar cuando el usuario pregunta* de *actualizar cuando el hecho ocurre*, y es la que determina si dos puestos de trabajo ven o no lo mismo al mismo tiempo.

La última pieza es la identidad. En un sistema con puestos de trabajo que tienen permisos distintos, la práctica establecida consiste en delegar la autenticación en un proveedor de identidad que implemente OpenID Connect sobre OAuth 2.0 (Sakimura et al., 2023), en lugar de programar dentro de la propia aplicación el almacenamiento de contraseñas, la expiración de sesiones y el límite de intentos fallidos. El proveedor emite un token firmado y con vencimiento que transporta el rol del usuario; la aplicación se limita a validar esa firma y a comprobar, **en el servidor**, qué operaciones admite ese rol. La interfaz solo oculta lo que no corresponde, pero no es la que autoriza.

Sobre esa base, el dominio cuenta con diversas soluciones tecnológicas, desde plataformas comerciales en la nube hasta herramientas de código abierto. A continuación se relevan tres soluciones representativas y el proceso manual que Max Pizzas emplea en la actualidad, señalando en cada caso qué resuelven y qué dejan sin resolver frente a las necesidades específicas del negocio.

Solución	Alcance	Qué resuelve	Qué deja sin resolver	Fuente citable
Fudo (plataforma comercial en la nube)	Plataforma integral para gastronomía: comandas digitales o impresas, control de stock e insumos, caja, reportes, tienda online y delivery. Declara más de 35.000 negocios activos en Latinoamérica	Todos los pedidos — salón, mostrador, carta QR, mensajería y aplicaciones de reparto— entran a un mismo sistema y la comanda llega a cocina sin papel	Modelo de suscripción y datos alojados en la nube del proveedor, sin posibilidad de autoalojamiento; el flujo de trabajo y los roles son los que define la plataforma, no los del proceso propio del negocio	Fudo (2026)

Solución	Alcance	Qué resuelve	Qué deja sin resolver	Fuente citable
Loyverse KDS (aplicación comercial gratuita)	Pantalla de cocina para tabletas iOS y Android, conectada al punto de venta del mismo proveedor; encabezados con código de color según el tiempo de espera	La comanda llega al instante a la pantalla de cocina, con aviso sonoro y marcado de artículos o pedidos completos, sin costo de licencia ni tickets en papel	Depende del punto de venta y de la nube del mismo proveedor; funciona solo como aplicación de tableta, no como sistema web propio; no cubre el flujo ni los roles del proceso local y los datos quedan fuera del control del negocio	Loyverse (2026)
Open Source Point of Sale (OSPOS) (código abierto, autoalojable)	Punto de venta web de código abierto, versión 3.4, escrito en PHP sobre CodeIgniter 4 con MySQL: artículos, ventas, facturación, gastos, caja, informes y control de usuarios con permisos	Base de datos y operaciones de creación, consulta, modificación y eliminación bajo control del negocio, con usuarios y permisos, sin costo de licencia	No incluye pantalla de cocina ni seguimiento del estado del pedido en tiempo real entre recepción y cocina; sus reglas son las de un punto de venta genérico y adaptarlo exigiría trabajar sobre un stack distinto	Open Source Point of Sale (2026)
Proceso manual actual de Max Pizzas	Uso interno: cuadernos de anotación y traspaso de palabra a cocina	Registro mínimo del pedido mientras no hay sistema	Sin trazabilidad, sin visibilidad del estado en tiempo real, sin registro de la disponibilidad de productos y sin control de acceso	Entrevista a Herlinda Duran, dueña de Max Pizzas, 2026

Tabla 2. Soluciones informáticas existentes en el dominio. Fuente: elaboración propia.

De las tres soluciones examinadas, Fudo y Loyverse KDS resuelven la llegada de la comanda a cocina y su seguimiento, pero lo hacen dentro de sus propios servicios: los datos residen en la nube del proveedor y el flujo de trabajo y los roles son los que define la plataforma. OSPOS sí admite autoalojamiento y control de usuarios, pero la versión revisada no contempla la pantalla de cocina ni el seguimiento del estado del pedido en tiempo real entre recepción y cocina.

En Max Pizzas la necesidad identificada se concentra precisamente ahí: que recepción y cocina compartan el estado actualizado de cada pedido. Sobre ese hueco se plantea **Max Pizzapp** como un sistema web propio, delimitado al local central de Villa Fátima, cuyo desarrollo permitirá ajustar el registro, la preparación y la entrega de pedidos al proceso de trabajo relevado en el negocio, integrando las tecnologías del diplomado.

1.2 Descripción del problema

Max Pizzas es una pizzería de la ciudad de Tarija que opera con un local central en la zona de Villa Fátima. En la actualidad, la toma y coordinación de sus pedidos se realiza de forma **manual**: la recepción anota cada pedido en papel y **se lo pasa de palabra** a la cocina, que prepara según lo que va escuchando o recibiendo. No existe ningún medio que muestre el estado de cada pedido (pendiente, en preparación, listo) de forma compartida y actualizada.

El local atiende de **18:00 a 23:30**, y dentro de ese horario la demanda se concentra en una franja estrecha: entre las **19:30 y las 22:00** se atienden **entre 50 y 70 pedidos**, sumando el salón —que dispone de **8 mesas**—, los pedidos recibidos por redes sociales, en su mayoría por mensajería instantánea, y los de reparto a domicilio. Un pedido tarda normalmente **entre 12 y 15 minutos** desde que se toma hasta que se entrega.

Sobre ese proceso se observan tres puntos críticos, verificados con la responsable del negocio:

Primero, las demoras no son visibles para quien atiende. Como no existe un medio compartido que muestre el estado de cada pedido, la recepción no advierte que un pedido se ha estancado hasta que el cliente reclama. En la semana relevada se registraron **entre 6 y 7 pedidos demorados por día**.

Segundo, la falta de un producto se descubre demasiado tarde. La disponibilidad de la carta no se refleja en ningún registro compartido: un pedido puede permanecer en cola y recién al momento de prepararlo se advierte que no queda ese tipo de pizza. Entonces hay que comunicarse con el cliente, pedirle que elija otro producto y volver a empezar la preparación, lo que **añade entre 10 y 15 minutos** a la espera de ese pedido, sobre los 12 a 15 minutos habituales.

Tercero, pasar los pedidos de palabra produce errores y olvidos. Entre **2 y 3 pedidos por día** se preparan mal, se traspapelan o se olvidan, con el consiguiente malestar del cliente y la repetición del trabajo en cocina.

N.º	Punto crítico	Dato del contexto que lo evidencia
1	Las demoras en la preparación no son visibles para recepción: no hay un medio compartido que muestre el estado de cada pedido	Entre 6 y 7 pedidos demorados por día en la semana relevada, sobre un volumen de 50 a 70 pedidos en la franja de mayor demanda (19:30–22:00)
2	La falta de disponibilidad de un producto se detecta recién al momento de preparar el pedido, ya encolado	Cada caso añade de 10 a 15 minutos de espera al pedido afectado, frente a los 12 a 15 minutos que dura normalmente el proceso, y obliga a contactar al cliente para sustituir el producto
3	Los pedidos se toman y se pasan a cocina de palabra, sin un registro verificable	Entre 2 y 3 pedidos por día se preparan mal, se traspapelan o se olvidan

Tabla 3. Puntos críticos del proceso actual. Fuente: elaboración propia a partir de entrevista con la responsable del negocio.

Los tres datos fueron proporcionados por **Herlinda Duran**, dueña de Max Pizzas, en entrevista presencial realizada en el local central de la zona Villa Fátima el **18 de septiembre de 2026**, y corresponden a estimaciones de la propia responsable sobre la operación de la semana previa.

1.3 Planteamiento del problema

El proceso de toma y preparación de pedidos en la pizzería **Max Pizzas**, de la ciudad de Tarija, se realiza actualmente de forma **manual: los pedidos se anotan en papel y se pasan de palabra entre recepción y cocina**, lo que produce **entre 6 y 7 pedidos demorados por día y entre 2 y 3 pedidos mal preparados, traspapelados u olvidados por día**, sobre un volumen de 50 a 70 pedidos en la franja de mayor demanda. **No se cuenta con un medio compartido que refleje el estado de cada pedido ni la disponibilidad de los productos en tiempo real**, lo que impide a la recepción advertir las demoras cuando ocurren y obliga a descubrir la falta de un producto recién al momento de prepararlo, con 10 a 15 minutos adicionales de espera para el cliente.

1.4 Justificación del problema

Factibilidad técnica

El desarrollo de Max Pizzapp **se considera técnicamente viable** dentro del alcance definido para el módulo, con tecnologías disponibles y ya trabajadas en el diplomado: Node.js con Express para la API, PostgreSQL para la persistencia, Keycloak para la autenticación y autorización por rol, Flutter para la interfaz web, WebSockets para la sincronización en tiempo real, y Docker para la orquestación. Estas tecnologías permiten organizar el sistema en componentes diferenciados y responden a los requisitos descritos en el apartado 2.3.

El proceso que se automatiza pertenece al propio negocio y fue relevado mediante entrevista con su responsable, que autorizó el relevamiento; la carta y los datos de operación necesarios para construir y probar el sistema se obtendrán por esa misma vía. El alcance —un solo local, dos roles y el flujo de pedidos con seguimiento en tiempo real— es acotado y se estima realizable en el plazo del módulo. Para la publicación del sistema se ha previsto una infraestructura con capa gratuita permanente; sus condiciones de uso y sus límites se verificarán durante la fase de despliegue, en el apartado 2.9.

Pertinencia

El problema es actual y concreto: Max Pizzas coordina hoy sus pedidos de forma manual, lo que ocasiona demoras no advertidas y errores de preparación en un negocio cuyo servicio depende de la rapidez, y en una franja horaria —de 19:30 a 22:00— en la que se concentra la mayor parte de la demanda. Atender esta coordinación en tiempo real es pertinente para el negocio y apropiado como Trabajo Final, por tratarse de un dominio conocido que permite integrar de forma competente lo aprendido en el diplomado. **La existencia del problema fue validada con Herlinda Duran, dueña de Max**

Pizzas, quien proporcionó los datos de operación consignados en el apartado 1.2 y autorizó el relevamiento del proceso.

Impacto esperado

Con Max Pizzapp desplegado se espera que recepción y cocina compartan una visión única y actualizada del estado de cada pedido. Sobre los tres puntos críticos del apartado 1.2, el impacto esperado es: que las demoras dejen de ser invisibles para recepción —hoy de 6 a 7 pedidos por día—, porque el cambio de estado se propagará a las pantallas conectadas sin que el usuario tenga que consultar a cocina; que la falta de un producto se conozca antes de tomar el pedido y no al momento de prepararlo, evitando los 10 a 15 minutos adicionales de espera y la llamada al cliente; y que disminuyan los 2 a 3 pedidos diarios mal preparados u olvidados, al sustituir el traspaso de palabra por un registro compartido y trazable.

Conviene distinguir dos planos. El **cumplimiento de los requisitos técnicos** —la propagación en menos de 2 segundos, la validación por rol, el aviso ante la caída del canal— se verificará con los casos de prueba del apartado 2.8. La **reducción efectiva de demoras y errores en la operación del negocio**, en cambio, exigirá una comparación posterior con el proceso manual y no puede darse por alcanzada en esta etapa: se plantea como una nueva consulta a la responsable del negocio tras un período de uso, y se recoge entre las recomendaciones del apartado 3.2.

1.5 Objetivos

1.5.1 Objetivo general

Desarrollar un sistema web de gestión de pedidos con sincronización en tiempo real del estado de preparación, **para el personal de recepción y cocina** de la pizzería **Max Pizzas**, en su **local central de la zona Villa Fátima de la ciudad de Tarija**, durante la gestión 2026.

1.5.2 Objetivos específicos

1. Analizar los requisitos del proceso de toma y preparación de pedidos de Max Pizzas.
2. Diseñar la arquitectura y el modelo de datos de Max Pizzapp.
3. Implementar los módulos de recepción y cocina con sincronización en tiempo real del estado del pedido.
4. Aplicar mecanismos de autenticación y autorización por rol mediante Keycloak.
5. Ejecutar el plan de pruebas del flujo de pedidos y de la sincronización en tiempo real.
6. Desplegar Max Pizzapp en una dirección pública accesible.

N.º	Objetivo específico	Evidencia en el Capítulo 2	Conclusión en el Capítulo 3
1	Analizar los requisitos del proceso de toma y preparación de pedidos de Max Pizzas	2.3 Análisis de requisitos	3.1, conclusión 1
2	Diseñar la arquitectura y el modelo de datos de Max Pizzapp	2.4 Diseño de la solución	3.1, conclusión 2
3	Implementar los módulos de recepción y cocina con sincronización en tiempo real del estado del pedido	2.6 Implementación	3.1, conclusión 3
4	Aplicar mecanismos de autenticación y autorización por rol mediante Keycloak	2.7 Seguridad	3.1, conclusión 4
5	Ejecutar el plan de pruebas del flujo de pedidos y de la sincronización en tiempo real	2.8 Pruebas	3.1, conclusión 5
6	Desplegar Max Pizzapp en una dirección pública accesible	2.9 Despliegue	3.1, conclusión 6

Tabla 4. Correspondencia objetivo – evidencia – conclusión. Fuente: elaboración propia.

2.2 Metodología de desarrollo

Marco de trabajo adoptado: Kanban, con tablero de cuatro columnas —*Por hacer, En curso, En pruebas y Hecho*— y **límite de trabajo en curso de una tarjeta.**

Justificación de la elección. Los Lineamientos Técnicos exigen no solo declarar un marco de trabajo, sino justificar su elección y evidenciar su aplicación mediante iteraciones definidas, un tablero o *backlog* y un cronograma efectivamente ejecutado. La elección de Kanban responde a tres características concretas de este proyecto:

1. **Se desarrolla individualmente.** Scrum define roles —dueño de producto, *Scrum Master* y equipo de desarrollo— y ceremonias de equipo que no tienen a quién asignarse ni cómo evidenciarse en un trabajo de una sola persona. Kanban, en cambio, organiza el flujo de tareas sin requerir esa estructura.
2. **El avance queda registrado de forma verificable.** El tablero muestra en qué etapa está cada funcionalidad, y el historial del repositorio acompaña ese avance con los cambios efectivamente incorporados.
3. **El límite de trabajo en curso protege el plazo.** Con una sola tarjeta en curso a la vez, cada funcionalidad se termina y se prueba antes de abrir la siguiente, lo que evita llegar a la última semana con varias funcionalidades a medias.

Ciclo de trabajo de cada tarjeta. El proyecto adopta un ciclo fijo para todas las tarjetas: plan escrito y aprobado → implementación de lo aprobado → pruebas de extremo a extremo → revisión del resultado → incorporación al repositorio. El plan de cada tarjeta se versiona en el propio repositorio, en docs/planes/, y se incorpora **antes** que el código correspondiente; al cerrarse la tarjeta, ese mismo archivo recoge la evidencia de las pruebas ejecutadas y los *commits* que la cerraron. Al cierre de la Semana 1 el procedimiento está definido y la primera tarjeta tiene su plan redactado; su

aplicación completa —construcción, pruebas y cierre— corresponde a las semanas siguientes, y de ahí se alimentará el apartado 2.8.

Correspondencia con el cronograma del módulo. Las cuatro semanas del cronograma académico se utilizan para organizar las entregas previstas. La Tabla 5 conserva las denominaciones que fija la plantilla del módulo; la gestión cotidiana del proyecto, en cambio, se realiza mediante el flujo continuo de Kanban y no mediante ceremonias de equipo.

Sprint	Semana	Objetivo del sprint	Requisitos incluidos
Sprint 0	1 (11–20 sep)	Requisitos, arquitectura y repositorio: perfil de proyecto, modelo de datos y repositorio inicializado con su historial	— (definición de alcance: RF-01 a RF-13, RNF-01 a RNF-05)
Sprint 1	2 (21–27 sep)	Vertical funcional desplegada: autenticación por rol y una pantalla completa contra la API y la base de datos, accesible en dirección pública	RF-01, RF-02, RF-06 · RNF-02, RNF-04
Sprint 2	3 (28 sep – 4 oct)	Autenticación, CRUD completo y pruebas: ciclo de vida completo del pedido con sincronización en tiempo real y evidencia de pruebas	RF-03, RF-04, RF-05, RF-07, RF-09, RF-13 · RNF-01, RNF-03, RNF-05
Sprint 3	4 (5–11 oct)	Estabilización del despliegue público ya publicado en la Semana 2, monitoreo, endurecimiento de seguridad, manuales y cierre documental	RF-08, RF-10, RF-11, RF-12 (según disponibilidad) · RNF-02

Tabla 5. Plan de sprints. Fuente: elaboración propia.

Herramienta de tablero utilizada: GitHub Projects · **Enlace:** <https://github.com/users/jorgeluisgutierrezrivera/projects/2> (proyecto público) · **Límite de trabajo en curso: 1 tarjeta**

El tablero tiene las cuatro columnas declaradas —Por hacer, En curso, En pruebas y Hecho—, un campo de prioridad MoSCoW por tarjeta y 23 tarjetas que cubren desde la apertura del repositorio hasta el cierre documental. Al cierre de la Semana 1 hay 6 tarjetas en Hecho, 1 en curso y 16 por hacer.

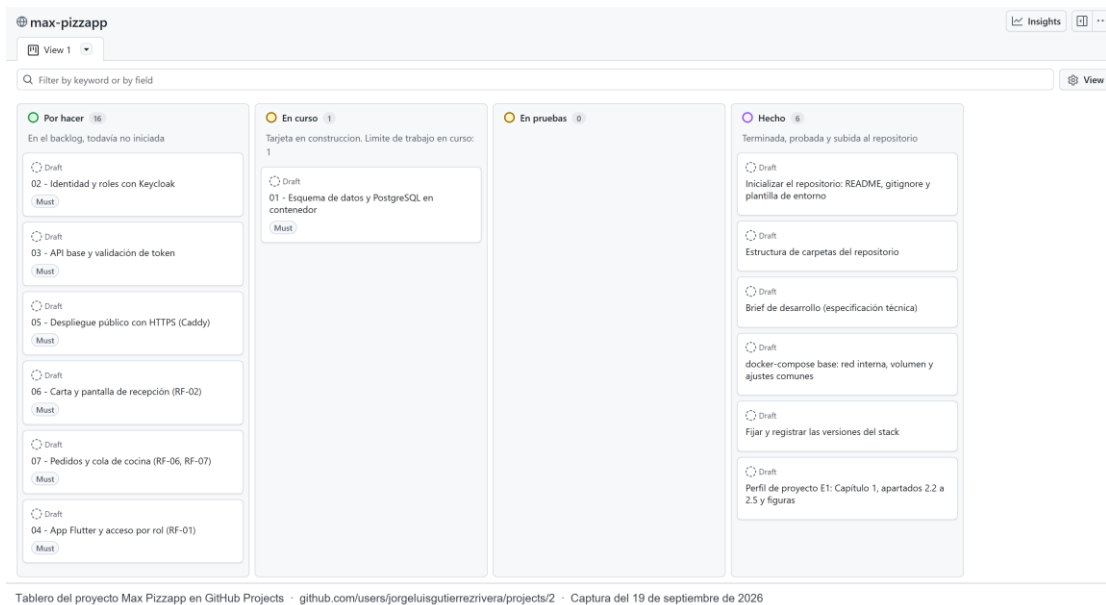


Figura 1. Tablero de tareas al cierre de la Semana 1. Fuente: elaboración propia.

Uso de asistentes de inteligencia artificial

Durante el desarrollo se emplearon asistentes de inteligencia artificial como herramientas de apoyo, y se declaran aquí por transparencia. **Claude Code**, de Anthropic, se utilizó en tareas de programación: redacción de borradores de documentación técnica, exploración de alternativas de diseño, revisión de la configuración de contenedores, elaboración de diagramas y generación de casos de prueba. **ChatGPT**, de OpenAI, se utilizó como apoyo en la revisión de coherencia documental, el contraste del contenido con las plantillas institucionales, la organización de apartados y la revisión de redacción.

Las decisiones de alcance, de arquitectura y de modelo de datos, así como la totalidad de los *commits* del repositorio, fueron tomadas y ejecutadas por el autor, que revisó y validó cada resultado antes de incorporarlo y mantiene la responsabilidad sobre el sistema y el documento presentados. El criterio aplicado fue que ningún fragmento del sistema ni del documento se incorpora si el autor no puede explicarlo y ejecutarlo por sí mismo, condición necesaria para la defensa técnica.

Evidencia de aplicación de la metodología

Evidencia	Dónde se verifica
Iteraciones definidas y ejecutadas	Tabla 5 y capturas fechadas del tablero al cierre de cada semana
Tablero con límite de trabajo en curso	https://github.com/users/jorgeluisgutierrezrivera/projects/2 · Figura 1
Cronograma efectivamente ejecutado	Historial de <i>commits</i> del repositorio, en fechas distintas y con mensajes que describen el cambio
Planificación previa a la construcción	Carpeta docs/planes/ del repositorio: un plan por tarjeta, incorporado antes que su código
Decisiones técnicas y su motivo	Bitácora de decisiones del proyecto, en formato de registro breve por decisión
Dificultades encontradas y su resolución	Base de errores del proyecto: causa, solución y prevención de cada incidencia (insumo del apartado 2.6)

Tabla 5.b Evidencia de aplicación de la metodología. Fuente: elaboración propia.

2.3 Análisis de requisitos

2.3.1 Actores y perfiles de usuario

ID	Actor	Descripción	Qué puede hacer	Qué no puede hacer
A-01	Recepción	Personal que atiende el mostrador de la pizzería: recibe al cliente, toma el pedido y lo entrega cuando está listo. Opera desde una tableta o un equipo de escritorio.	Iniciar sesión con sus credenciales · Consultar la carta de productos disponibles · Registrar un pedido y su observación · Ver el estado de todos los pedidos activos, actualizado en vivo · Cancelar un pedido mientras no esté listo · Marcar como entregado un pedido que está listo · Marcar un producto como agotado	Cambiar el estado de un pedido a <i>en preparación</i> o a <i>listo</i> : es competencia exclusiva de cocina · Marcar como entregado un pedido que aún no está listo · Crear, editar o eliminar productos de la carta, ni modificar precios: solo puede marcar un producto como agotado · Eliminar pedidos ni alterar el historial de estados
A-02	Cocina	Personal de preparación. Opera desde una tableta o un monitor fijo en la zona de cocina, de pie y sin teclado.	Iniciar sesión con sus credenciales · Ver la cola de pedidos entrantes ordenada por hora de llegada, actualizada en vivo · Consultar el detalle y la observación de cada pedido · Pasar un pedido a <i>en preparación</i> · Marcar un pedido como <i>listo</i> · Marcar un producto como agotado	Registrar pedidos · Marcar pedidos como entregados · Cancelar pedidos · Crear, editar o eliminar productos de la carta, ni modificar precios: solo puede marcar un producto como agotado · Consultar los datos de contacto del cliente

Tabla 6. Actores del sistema. Fuente: elaboración propia.

Autenticación de ambos perfiles. La identidad y las credenciales las gestiona el proveedor de identidad (Keycloak): las contraseñas se almacenan con función de *hash* y el sistema no mantiene tabla propia de usuarios. El acceso se resuelve **por redirección al proveedor** —OpenID Connect con el flujo *Authorization Code* y PKCE—, de modo que las credenciales se introducen en Keycloak y **la aplicación nunca las recibe**; esta obtiene a cambio un token firmado, con vigencia de 60 minutos, que incorpora el rol del usuario. **Si la autenticación falla**, es el propio proveedor el que rechaza el acceso con un mensaje que no revela si el usuario existe, y el que aplica el límite de intentos consecutivos. La pertenencia al rol se verifica **en el servidor en cada petición**, no solo al entrar a la pantalla: la interfaz oculta lo que el rol no puede hacer, pero es el servidor el que lo impide.

Decisión de alcance. El módulo admite un máximo de dos roles diferenciados. Las capacidades de administración de la carta y de consulta del historial se declaran fuera de alcance en la Tabla 9, y se recogen como trabajo futuro en el apartado 3.2.

2.3.2 Requisitos funcionales

ID	Requisito funcional	Criterios de aceptación	MoSCoW
RF-01	Como usuario del sistema, quiero iniciar sesión con mis credenciales, para acceder a las funciones de mi rol.	CA-01.1 Dado un usuario con credenciales válidas, cuando las introduce en el proveedor de identidad, entonces el sistema recibe su token, lo valida y lo dirige a la pantalla correspondiente a su rol. · CA-01.2 Dado un usuario con credenciales inválidas, cuando intenta acceder, entonces el proveedor rechaza el acceso sin emitir token y el usuario permanece fuera del sistema, que no muestra ninguna pantalla de su rol.	Must
RF-02	Como recepción, quiero registrar un pedido seleccionando productos de la carta, para enviarlo a cocina.	CA-02.1 Dada una carta con productos disponibles, cuando la recepción selecciona productos y confirma, entonces el sistema crea el pedido en estado <i>pendiente</i> y lo envía a cocina. · CA-02.2 Dado un pedido sin productos, cuando la recepción intenta confirmarlo, entonces el sistema no lo crea y muestra "Agregue al menos un producto".	Must
RF-03	Como recepción, quiero ver el estado de cada pedido actualizado en tiempo real, para conocer su avance sin consultar a cocina.	CA-03.1 Dado un pedido activo, cuando cocina cambia su estado, entonces la pantalla de recepción refleja el nuevo estado en menos de 2 segundos sin recargar. · CA-03.2 Dada una caída del canal en tiempo real, cuando se pierde la conexión, entonces el sistema muestra un aviso y permite recargar para ver el estado actual.	Must
RF-04	Como recepción, quiero recibir un aviso cuando un pedido pase a <i>listo</i> , para entregarlo de inmediato.	CA-04.1 Dado un pedido en preparación, cuando cocina lo marca <i>listo</i> , entonces recepción recibe un aviso visible y el pedido aparece destacado. · CA-04.2 Dado que no hay pedidos listos, cuando la recepción abre su pantalla, entonces se muestra el estado vacío sin avisos pendientes.	Must
RF-05	Como recepción, quiero marcar un pedido como entregado, para cerrar su atención.	CA-05.1 Dado un pedido en estado <i>listo</i> , cuando la recepción lo marca como entregado, entonces el sistema registra la entrega y lo retira de la lista activa. · CA-05.2 Dado un pedido que aún no está <i>listo</i> , cuando la recepción intenta entregarlo, entonces el sistema lo impide y muestra "El pedido aún no está listo".	Must
RF-06	Como cocina, quiero ver en tiempo real los pedidos entrantes ordenados por llegada, para prepararlos en orden.	CA-06.1 Dado un nuevo pedido confirmado por recepción, cuando se crea, entonces aparece en la pantalla de cocina en menos de 2 segundos, al final de la cola. · CA-06.2 Dado que no hay pedidos pendientes, cuando cocina abre su pantalla, entonces se muestra el estado vacío.	Must
RF-07	Como cocina, quiero cambiar el estado de un pedido (<i>en preparación / listo</i>), para informar el avance a recepción.	CA-07.1 Dado un pedido pendiente, cuando cocina lo pasa a <i>en preparación</i> y luego a <i>listo</i> , entonces el sistema guarda cada cambio y lo propaga a recepción en tiempo real. · CA-07.2 Dado un pedido ya entregado, cuando cocina intenta cambiar su estado, entonces el sistema lo impide por tratarse de un estado final.	Must
RF-08	Como usuario del sistema, quiero cerrar sesión y que otro compañero inicie la suya en el mismo dispositivo, para que cada acción quede registrada a nombre de quien la ejecuta.	CA-08.1 Dado un usuario con sesión iniciada, cuando cierra sesión, entonces el sistema descarta el token, vuelve a la pantalla de acceso y el siguiente usuario inicia su propia sesión. · CA-08.2 Dado un token expirado, cuando el usuario intenta una operación, entonces el sistema la rechaza y lo devuelve a la pantalla de acceso.	Should

ID	Requisito funcional	Criterios de aceptación	MoSCoW
RF-09	Como recepción, quiero cancelar un pedido antes de que esté listo, para corregir errores de toma.	CA-09.1 Dado un pedido en <i>pendiente</i> o <i>en preparación</i> , cuando la recepción lo cancela indicando un motivo, entonces el sistema lo marca <i>cancelado</i> y lo notifica a cocina en tiempo real. · CA-09.2 Dado un pedido en <i>listo</i> o <i>entregado</i> , cuando se intenta cancelar, entonces el sistema lo impide.	Should
RF-10	Como cocina, quiero ver cuánto tiempo lleva esperando cada pedido de la cola, para priorizar los más antiguos.	CA-10.1 Dado un pedido en la cola, cuando transcurre el tiempo, entonces la tarjeta muestra los minutos transcurridos desde su registro, actualizados sin recargar. · CA-10.2 Dado un pedido que supera el umbral de espera definido, cuando se muestra en la cola, entonces se destaca con una etiqueta de texto además del color.	Could
RF-11	Como recepción, quiero agregar una observación a un pedido (por ejemplo, "sin cebolla"), para comunicar indicaciones a cocina.	CA-11.1 Dado un pedido en registro, cuando la recepción agrega una nota, entonces la nota se muestra en cocina junto al pedido. · CA-11.2 Dada una nota que excede el máximo permitido, cuando se intenta guardar, entonces el sistema la rechaza e indica el límite.	Could
RF-12	Como recepción, quiero visualizar un comprobante del pedido, para mostrarlo al cliente.	CA-12.1 Dado un pedido registrado, cuando la recepción abre su comprobante, entonces el sistema muestra productos, cantidades, total y número de pedido. · CA-12.2 Dado un pedido inexistente, cuando se solicita su comprobante, entonces el sistema muestra "Pedido no encontrado".	Could
RF-13	Como cocina o recepción, quiero marcar un producto como agotado, para que deje de ofrecerse en la carta y no se tomen pedidos que no se pueden preparar.	CA-13.1 Dado un producto disponible, cuando cocina o recepción lo marca como agotado, entonces el sistema lo deja de mostrar como seleccionable en la carta y el cambio llega a las pantallas conectadas en menos de 2 segundos. · CA-13.2 Dado un producto agotado que ya figura en un pedido en curso, cuando se marca como agotado, entonces el sistema conserva ese pedido sin alterarlo y solo impide nuevas selecciones del producto.	Should

Tabla 7. Requisitos funcionales, criterios de aceptación y prioridad. Fuente: elaboración propia.

Total de requisitos: 13 · Must have: 7 · Porcentaje Must have: 54 % (debe ser ≤ 60 %)

2.3.3 Requisitos no funcionales

ID	Tipo	Enunciado	Métrica de verificación
RNF-01	Rendimiento	Los cambios de estado deben reflejarse en las pantallas conectadas sin intervención del usuario, y el listado principal debe cargar con fluidez en la carga habitual del local	Propagación de un cambio de estado en menos de 2 segundos ; carga del listado principal en menos de 2 segundos con 50 pedidos activos y 5 usuarios concurrentes
RNF-02	Seguridad	El acceso se autentica contra el proveedor de identidad, las comunicaciones viajan cifradas y la autorización por rol se comprueba en el servidor	Autenticación OpenID Connect con Keycloak 26.7; token JWT con expiración de 60 minutos ; HTTPS en todo el tráfico público; contraseñas con <i>hash</i> gestionadas por el proveedor; el 100 % de las rutas protegidas valida el token y los permisos que exige la ruta ; las operaciones que cambian el estado de un pedido comprueban además que la transición solicitada sea válida

ID	Tipo	Enunciado	Métrica de verificación
RNF-03	Usabilidad	La tarea principal debe completarse en pocos pasos y toda vista con datos debe informar su situación al usuario	Registrar un pedido en 3 pasos o menos ; 4 estados implementados (cargando, con datos, vacío y error) en todas las vistas que consumen datos
RNF-04	Compatibilidad	El sistema debe funcionar en los navegadores del local, tanto en equipo de escritorio como en tableta	Chrome y Edge, últimas 2 versiones ; interfaz adaptable verificada a 1366 px (escritorio) y 768 px (tableta) , sin desplazamiento horizontal
RNF-05	Disponibilidad	El sistema debe estar disponible durante el horario de atención y degradarse de forma controlada si cae el canal en tiempo real	Disponible durante todo el horario de atención del local, de 18:00 a 23:30 ; ante caída del canal, aviso visible al usuario en menos de 10 segundos y opción de recarga manual para consultar el estado actual

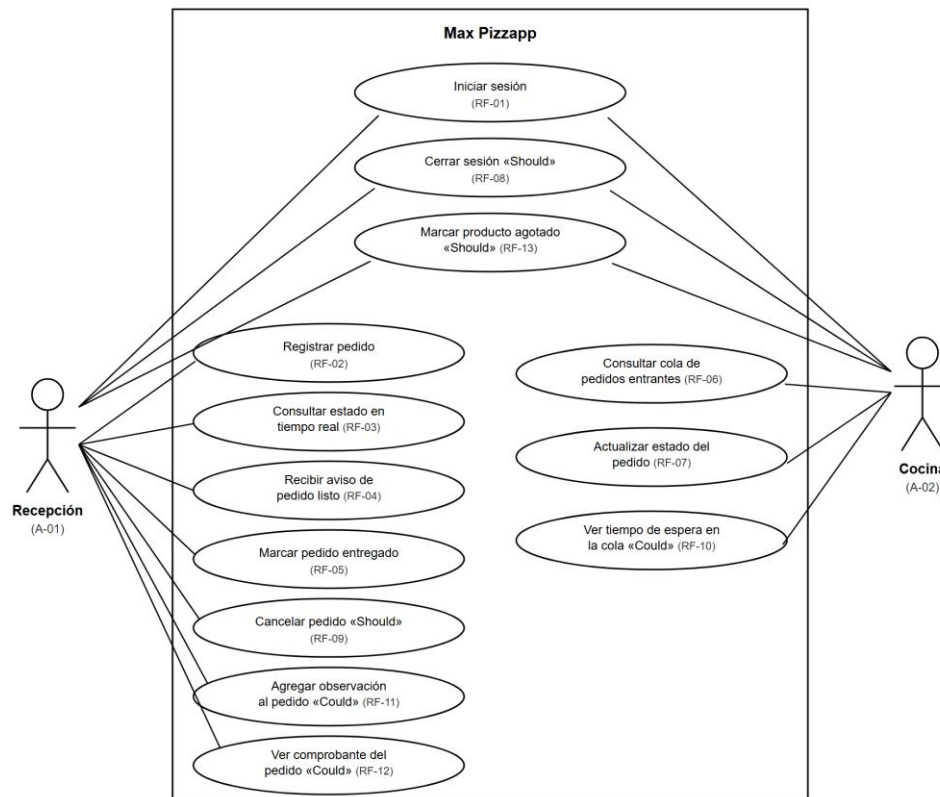
Tabla 8. Requisitos no funcionales. Fuente: elaboración propia.

2.3.4 Requisitos declarados fuera de alcance

Capacidad excluida	MoSCoW	Justificación de la exclusión
Operación multi-sede: sucursales de Villa Avaro y Tabladita, con aislamiento de datos por local	Won't	El proyecto se acota a un local piloto por acuerdo de la tutoría T1. El aislamiento de datos por sede añadiría complejidad sin aportar a la sincronización en tiempo real, que es el eje del trabajo. Queda como trabajo futuro en el apartado 3.2
Control de inventario: existencias por día y local, descuento de stock al confirmar el pedido y bloqueo de venta sin stock	Won't	Formaba parte del planteamiento original y se recortó en la tutoría T1. Exige un dominio propio —existencias, mermas, reposición— que no interviene en la coordinación entre recepción y cocina, que es el eje del trabajo. La necesidad inmediata que cubría —no ofrecer lo que no se puede preparar— se resuelve con RF-13
Control de caja y ventas: cobros, arqueo y reportes de venta	Won't	Es un dominio propio, con sus reglas y su normativa, que excede el plazo de cuatro semanas del módulo y no interviene en la coordinación entre recepción y cocina
Módulo de despacho como rol independiente	Won't	En el local piloto la entrega la realiza la propia recepción en mostrador. Crear un tercer rol multiplicaría pantallas, reglas de autorización y casos de prueba sin corresponder a la operación real
Gestión de la carta desde la aplicación (alta, edición y baja de productos) por un rol administrador	Could	Requiere un tercer rol diferenciado, por encima del máximo admitido. Durante el piloto la carta se carga en la base de datos con los datos iniciales. Queda como mejora recomendada en el apartado 3.2
Consulta del historial del día por un rol administrador	Could	Los datos quedan registrados en la entidad <code>historial_estado</code> , de modo que la capacidad puede añadirse después sin cambiar el modelo; se excluye de esta versión por la misma limitación de roles

Tabla 9. Capacidades declaradas fuera de alcance. Fuente: elaboración propia.

2.3.5 Casos de uso



Entre paréntesis, el requisito funcional que origina cada caso de uso. Las marcas «Should» y «Could» indican la prioridad MoSCoW; los casos sin marca son Must have. La autenticación y la gestión de credenciales las provee Keycloak; el sistema no administra usuarios.

Figura 2. Diagrama de casos de uso del sistema. Fuente: elaboración propia.

CU-01 Iniciar sesión

Actor principal: Recepción o Cocina · **Precondición:** El usuario tiene credenciales activas en el proveedor de identidad y el sistema está accesible.

Flujo principal: 1. El usuario abre la dirección del sistema. 2. La aplicación lo redirige al proveedor de identidad. 3. El usuario introduce allí su usuario y su contraseña. 4. El proveedor lo devuelve a la aplicación con un código de autorización de un solo uso. 5. La aplicación canjea ese código por un token firmado que incorpora el rol. 6. El sistema valida el token y dirige al usuario a la pantalla de su rol: toma de pedidos si es recepción, cola de preparación si es cocina.

Flujos alternativos: A1. En el paso 3, si las credenciales son inválidas, el proveedor rechaza el acceso sin indicar cuál de los dos datos falló y permite reintentar; la aplicación no llega a recibir ningún código. **A2.** Superado el número de intentos permitido, el proveedor bloquea temporalmente la cuenta e informa que debe esperar antes de volver a intentarlo.

Postcondición: El usuario queda autenticado, con un token vigente por 60 minutos y su rol determinado.

CU-02 Registrar pedido

Actor principal: Recepción · **Precondición:** La recepción inició sesión; la carta tiene productos disponibles.

Flujo principal: 1. La recepción abre “Nuevo pedido”. 2. Selecciona productos de la carta y sus cantidades. 3. Indica el nombre del cliente y, si corresponde, una observación. 4. Confirma el pedido. 5. El sistema valida la entrada en el servidor, crea el pedido en estado *pendiente* y registra sus líneas con el precio vigente. 6. El sistema emite el evento de pedido nuevo y la cola de cocina lo muestra en menos de 2 segundos.

Flujos alternativos: A1. En el paso 4, si no hay productos seleccionados, el sistema no crea el pedido y muestra “Agregue al menos un producto”. **A2.** Si alguno de los productos dejó de estar disponible entre la selección y la confirmación, el sistema rechaza la operación e indica qué producto debe retirarse.

Postcondición: El pedido queda registrado en estado *pendiente*, visible en la cola de cocina y en la lista de pedidos activos de recepción.

CU-03 Actualizar el estado del pedido

Actor principal: Cocina · **Precondición:** Cocina inició sesión; existe al menos un pedido en la cola.

Flujo principal: 1. Cocina abre la cola de pedidos, ordenada por hora de llegada. 2. Selecciona el pedido que comienza a preparar y lo pasa a *en preparación*. 3. El sistema valida en el servidor que el rol y la transición son admisibles, guarda el cambio y lo registra en el historial de estados. 4. El sistema propaga el nuevo estado a la pantalla de recepción en menos de 2 segundos. 5. Al terminar la preparación, cocina marca el pedido como *listo*. 6. Recepción recibe el aviso de pedido listo.

Flujos alternativos: **A1.** En el paso 3, si el pedido ya fue entregado o cancelado, el sistema rechaza el cambio por tratarse de un estado final. **A2.** Si el canal en tiempo real está caído, el sistema guarda igualmente el cambio y avisa al usuario de que la vista puede no estar actualizada, ofreciendo recargar.

Postcondición: El pedido queda en su nuevo estado, con el cambio registrado en el historial junto al usuario y la hora, y las pantallas conectadas reflejan el cambio.

2.4 Diseño de la solución

2.4.1 Arquitectura del sistema

Patrón arquitectónico adoptado: monolito modular en tres capas (cliente web, servidor de aplicación y base de datos), desplegado como conjunto de contenedores tras un proxy inverso, con un canal de eventos en tiempo real alojado en el mismo servidor de aplicación.

Justificación contra los requisitos:

- **RNF-01 (propagación < 2 s, 50 pedidos activos, 5 usuarios concurrentes).** La carga prevista es la de un solo local: un servidor de aplicación único la sostiene sin reparto entre servicios. El tiempo real se resuelve con un canal WebSocket alojado en ese mismo proceso, lo que evita el salto de red adicional que exigiría un servicio separado de notificaciones.
- **RF-03, RF-04 y RF-06 (estado en vivo en recepción y cocina).** El servidor de aplicación concentra las operaciones de escritura sobre la base de datos. El orden previsto es: guardar el cambio, confirmarlo en la base y solo entonces emitir el evento por el canal en tiempo real, de modo que no se anuncie un estado que no haya quedado persistido. Así la pantalla no necesita consultar periódicamente al servidor. Esto no elimina la posibilidad de que un cliente desconectado muestre información desactualizada: por eso el diseño contempla un aviso explícito al perderse el canal y la posibilidad de volver a consultar el estado actual por la API (RNF-05).
- **RNF-02 (autenticación, autorización por rol validada en el servidor, HTTPS).** Toda petición del cliente atraviesa el proxy inverso, que termina TLS, y luego el servidor de aplicación, que valida la firma del token y el rol antes de tocar la base de datos. El cliente nunca accede a la base de datos.
- **Plazo del módulo.** Un solo despliegue, una sola base de datos y una traza de depuración simple. Una arquitectura de microservicios multiplicaría

despliegues, red entre servicios y observabilidad sin aportar nada a los requisitos anteriores.

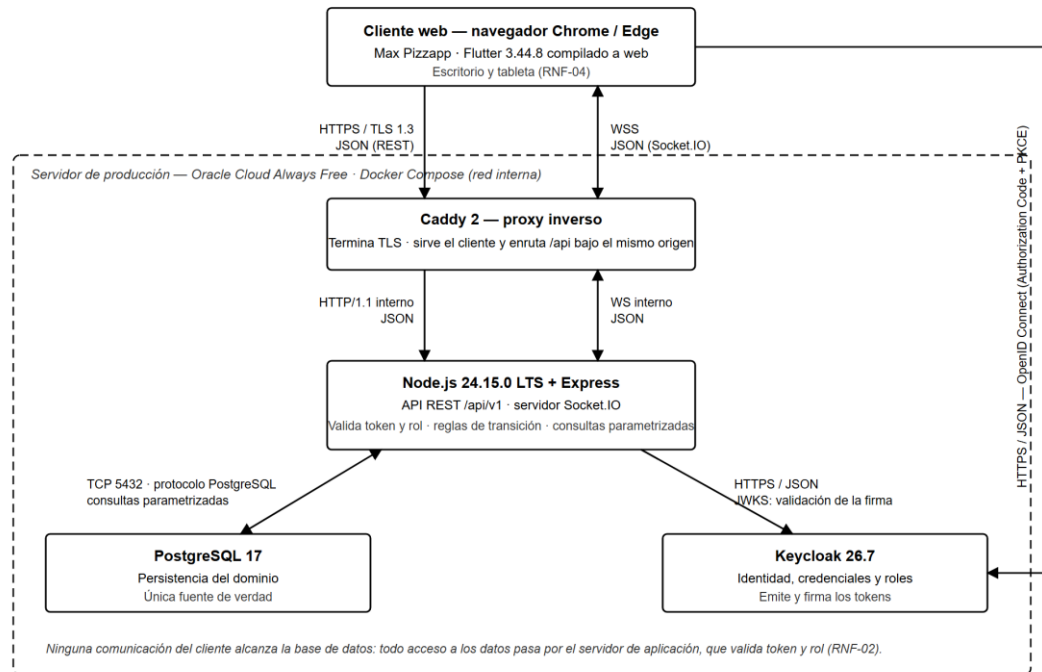


Figura 3. Arquitectura del sistema. Fuente: elaboración propia.

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
Ciente web	Presenta la carta, la toma de pedido y las colas de trabajo; recoge la entrada del usuario y la valida antes de enviarla. No decide reglas de negocio.	Flutter 3.44.8 (Dart 3.12.2), compilado a web	Proxy inverso	HTTPS / JSON (REST) y WSS / JSON (eventos)
Proxy inverso	Termina TLS, sirve los archivos estáticos del cliente y enruta /api y el canal de eventos hacia el servidor de aplicación, todo bajo el mismo origen.	Caddy 2	Ciente web · Servidor de aplicación · Proveedor de identidad	HTTPS y WSS hacia el exterior; HTTP/1.1 y WS en la red interna
Servidor de aplicación	Expone el contrato de la API, valida token y rol, aplica las reglas de transición de estado, ejecuta las consultas parametrizadas y emite los eventos en tiempo real.	Node.js 24.15.0 LTS + Express	Base de datos · Proveedor de identidad · Clientes conectados	TCP 5432 (protocolo PostgreSQL); HTTPS / JSON (JWKS); WSS / JSON (Socket.IO)
Base de datos	Guarda el estado del dominio y garantiza su integridad mediante claves foráneas, restricciones y tipos enumerados. Única fuente de verdad.	PostgreSQL 17	Servidor de aplicación	TCP 5432, protocolo nativo de PostgreSQL, consultas parametrizadas
Proveedor de identidad	Autentica usuarios, gestiona las contraseñas con hash, emite y	Keycloak 26.7	Ciente web · Servidor de aplicación	HTTPS / JSON (OpenID Connect, <i>Authorization Code</i> con PKCE) y HTTPS / JSON

Componente	Responsabilidad	Tecnología	Se comunica con	Protocolo / formato
	firma los tokens y define los roles del sistema.			(publicación de claves JWKS)

Tabla 10. Componentes y responsabilidades. Fuente: elaboración propia.

2.4.2 Modelo de datos

El modelo tiene **cinco entidades**, dentro del rango exigido. La identidad de los usuarios **no es una entidad del modelo**: la gestiona el proveedor de identidad, y en las tablas se conserva únicamente su identificador (sub, un UUID emitido por Keycloak) junto con una copia del nombre en el momento del hecho, para poder leer quién hizo qué sin consultar al proveedor en cada lectura.

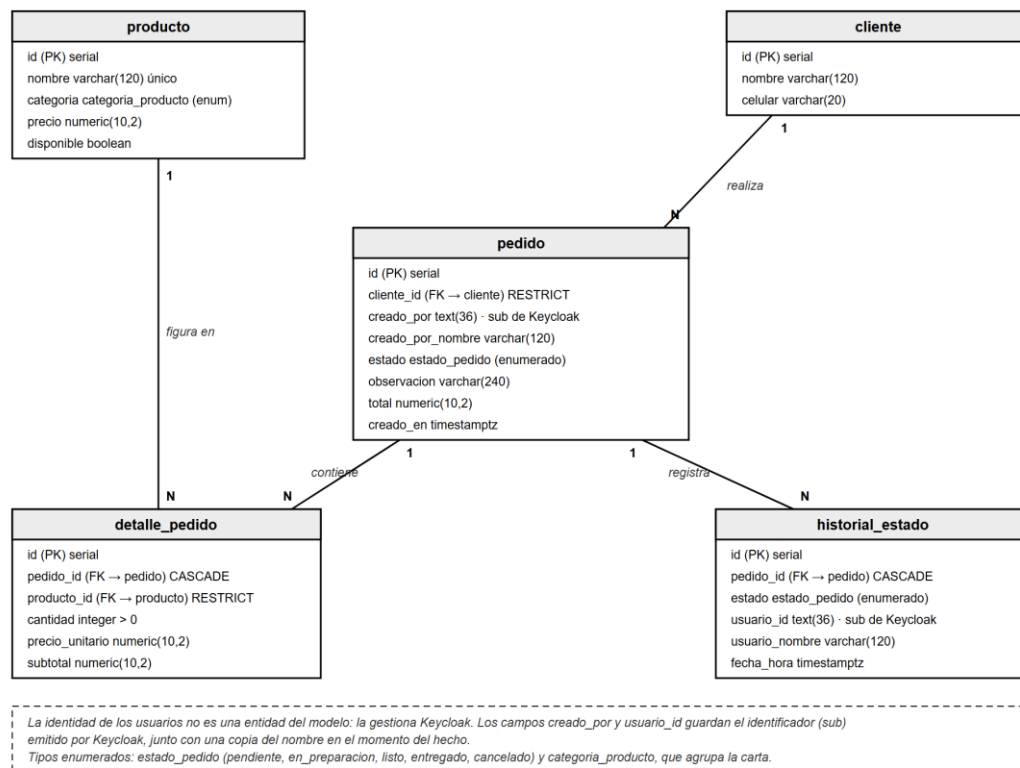


Figura 4. Modelo entidad-relación. Fuente: elaboración propia.

Relaciones: cliente 1—N pedido · pedido 1—N detalle_pedido · pedido 1—N historial_estado · producto 1—N detalle_pedido.

Reglas de integridad: las líneas de pedido y el historial de estados se eliminan **en cascada** con su pedido, por ser partes dependientes de él; en cambio no se permite eliminar un producto o un cliente que ya participa en un pedido (**RESTRICT**), lo que la API traduce a un código 409.

Conviene precisar cómo conviven esa cascada y la trazabilidad, que es uno de los objetivos del sistema: **el alcance del piloto no incluye ninguna operación de eliminación de pedidos accesible a los usuarios**. Ni recepción ni cocina pueden borrar un pedido —así se declara en la Tabla 6 y así lo refleja el contrato de la API, que no expone ninguna ruta DELETE sobre pedidos—; un pedido que no se llega a preparar se **cancela**, y la cancelación es un estado más que queda registrado en el historial. La cascada describe, por tanto, la integridad referencial del modelo ante una eliminación administrativa en la base de datos, no un camino previsto de la aplicación.

Diccionario de datos — entidad pedido

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	Clave primaria	Identificador del pedido
cliente_id	integer	—	No	Clave foránea → cliente.id, ON DELETE RESTRICT	Cliente que realiza el pedido
creado_por	text	36	Sí	—	Identificador (sub) del usuario de recepción que registró el pedido
creado_por_nombre	varchar	120	Sí	—	Nombre del usuario al momento del registro
estado	estado_pedido	—	Sí	Tipo enumerado; valor inicial pendiente	Estado actual del pedido
observacion	varchar	240	No	—	Indicación para cocina (por ejemplo, “sin cebolla”)
total	numeric	10,2	Sí	CHECK (total >= 0)	Importe del pedido: suma de los subtotales de sus líneas, conservado con el precio del momento
creado_en	timestampz	—	Sí	Valor por defecto: fecha y hora actuales	Momento del registro; ordena la cola de cocina

Tabla 11. Diccionario de datos de la entidad pedido. Fuente: elaboración propia.

Diccionario de datos — entidad detalle_pedido

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	Clave primaria	Identificador de la línea
pedido_id	integer	—	Sí	Clave foránea → pedido.id, ON DELETE CASCADE	Pedido al que pertenece la línea
producto_id	integer	—	Sí	Clave foránea → producto.id, ON DELETE RESTRICT	Producto solicitado
cantidad	integer	—	Sí	CHECK (cantidad > 0)	Unidades solicitadas
precio_unitario	numeric	10,2	Sí	CHECK (precio_unitario >= 0)	Precio del producto en el momento del pedido
subtotal	numeric	10,2	Sí	CHECK (subtotal >= 0)	Cantidad multiplicada por el precio unitario

Tabla 11.b Diccionario de datos de la entidad detalle_pedido. Fuente: elaboración propia.

Diccionario de datos — entidad producto

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	Clave primaria	Identificador del producto
nombre	varchar	120	Sí	Único	Nombre del producto en la carta
categoria	categoria_producto	—	Sí	Tipo enumerado	Agrupación con la que se muestra la carta: pizza, entrada, bebida o postre
precio	numeric	10,2	Sí	CHECK (precio >= 0)	Precio vigente
disponible	boolean	—	Sí	Valor por defecto: verdadero	Indica si el producto puede pedirse hoy

Tabla 11.c Diccionario de datos de la entidad producto. Fuente: elaboración propia.

Diccionario de datos — entidad cliente

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	Clave primaria	Identificador del cliente
nombre	varchar	120	Sí	—	Nombre con el que se anuncia el pedido
celular	varchar	20	No	—	Contacto para avisar que el pedido está listo

Tabla 11.d Diccionario de datos de la entidad cliente. Fuente: elaboración propia.

Diccionario de datos — entidad historial_estado

Campo	Tipo	Longitud	Obligatorio	Restricción	Descripción
id	serial	—	Sí	Clave primaria	Identificador del registro
pedido_id	integer	—	Sí	Clave foránea → pedido.id, ON DELETE CASCADE	Pedido al que corresponde el cambio
estado	estado_pedido	—	Sí	Tipo enumerado	Estado al que pasó el pedido
usuario_id	text	36	Sí	—	Identificador (sub) de quien ejecutó el cambio
usuario_nombre	varchar	120	Sí	—	Nombre del usuario al momento del cambio
fecha_hora	timestampz	—	Sí	Valor por defecto: fecha y hora actuales	Momento del cambio de estado

Tabla 11.e Diccionario de datos de la entidad historial_estado. Fuente: elaboración propia.

El modelo define dos tipos enumerados. `categoria_producto` agrupa la carta y `estado_pedido` admite los valores pendiente, en_preparacion, listo, entregado y cancelado. La base de datos garantiza que el valor sea uno de ellos; **qué transición es válida y qué rol puede ejecutarla se valida en el servidor**, que es la única capa confiable (RNF-02, RF-05, RF-07).

2.4.3 Diseño de la interfaz

Criterios de usabilidad y accesibilidad aplicados. Los siguientes criterios están incorporados al diseño de las pantallas y se recogen como tales en los wireframes; su implementación efectiva y su verificación corresponden a los apartados 2.6 y 2.8.

- 1. Tarea principal en tres pasos o menos (RNF-03).** Una vez abierta la pantalla “Nuevo pedido”, registrar un pedido son tres acciones: seleccionar productos, identificar al cliente y confirmar. La apertura de la pantalla corresponde a la navegación previa y no se cuenta como paso.
- 2. Los cuatro estados de toda vista con datos (RNF-03).** Cada pantalla que consume datos implementa *cargando*, *con datos*, *vacío* y *con error*, y distingue el error de red del error de negocio con mensajes distintos.

3. **Interfaz adaptable (RNF-04).** Disposición en una columna en tableta vertical y en dos columnas —carta y carrito— en escritorio, sin desplazamiento horizontal.
4. **Objetivos táctiles amplios.** Los controles de uso frecuente en cocina (avanzar estado, marcar listo) tienen un área mínima de 48 × 48 px, porque se operan de pie y con prisa.
5. **El color nunca es el único portador de información.** El estado del pedido se comunica con una etiqueta de texto además del color, para que una persona con dificultad para distinguir colores pueda operar el sistema.
6. **Contraste suficiente.** Texto y fondo mantienen una relación de contraste de al menos 4,5:1, verificable sobre todo en la pantalla de cocina, que se lee a distancia.
7. **Aviso explícito ante caída del canal en tiempo real (RNF-05).** La pantalla informa que los datos pueden no estar actualizados y ofrece recargar, en lugar de mostrar en silencio información vieja.

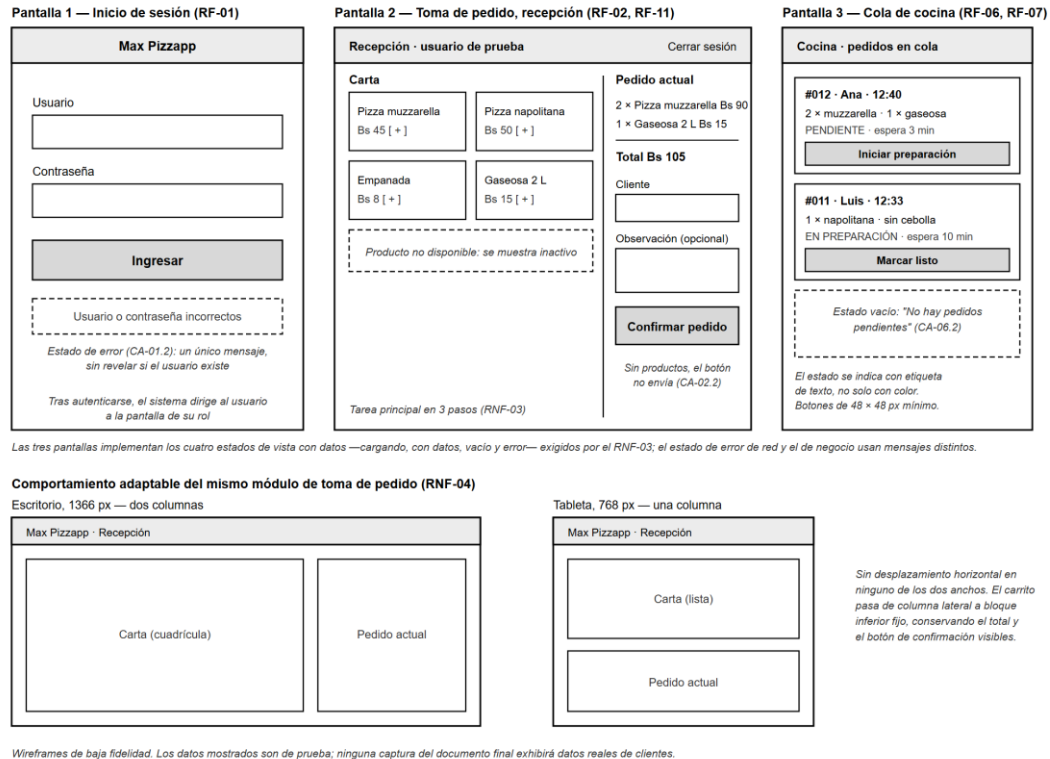


Figura 5. Wireframes de las pantallas principales. Fuente: elaboración propia.

2.4.4 Contrato de la interfaz de programación de aplicaciones

La autenticación no forma parte del contrato REST. Se resuelve por redirección al proveedor de identidad mediante OpenID Connect con el flujo *Authorization Code* y PKCE: el usuario introduce sus credenciales en Keycloak, no en la aplicación, y esta recibe un código que canjea por el token. En consecuencia, **la API nunca recibe ni maneja contraseñas**; se limita a validar la firma del token contra las claves publicadas por el proveedor.

Todas las rutas, salvo la de salud, exigen un token válido. El servidor valida la firma del token y los **permisos que exige cada ruta**, que no son los mismos en todas: la columna *RoI* de la Tabla 12 indica qué perfiles admite cada operación. Cuando la operación **modifica el estado de un pedido**, el servidor comprueba además que la transición solicitada sea válida desde el estado

actual. El contrato se publica bajo el prefijo /api/v1: la versión forma parte de la ruta para que un cambio incompatible pueda convivir con la versión previa.

Método	Ruta	Auth	Rol	Parámetros	2xx	Errores
GET	/api/v1/productos	sí	recepción	categoría, disponible	200 + lista de productos	401, 403
POST	/api/v1/pedidos	sí	recepción	cliente, observación, líneas (producto_id, cantidad)	201 + pedido creado	400, 401, 403, 409
GET	/api/v1/pedidos	sí	recepción, cocina	estado, page, limit	200 + lista de pedidos activos	401, 403
GET	/api/v1/pedidos/{id}	sí	recepción, cocina	id	200 + pedido con sus líneas	401, 403, 404
PATCH	/api/v1/pedidos/{id}/estado	sí	recepción, cocina	id + estado destino	200 + pedido actualizado	400, 401, 403, 404, 409
POST	/api/v1/pedidos/{id}/cancelación	sí	recepción	id + motivo	200 + pedido cancelado	400, 401, 403, 404, 409
PATCH	/api/v1/productos/{id}/disponibilidad	sí	recepción, cocina	id + disponible (booleano)	200 + producto actualizado	400, 401, 403, 404
GET	/api/v1/salud	no	—	—	200 + estado del servicio	503

Tabla 12. Contrato de la API. Fuente: elaboración propia.

Canal de eventos en tiempo real. Además del contrato REST, el servidor expone un canal WebSocket (Socket.IO) sobre el mismo origen, autenticado con el mismo token. Eventos emitidos: pedido:nuevo (carga útil: el pedido con sus líneas), pedido:estado (carga útil: identificador del pedido, estado anterior, estado nuevo y marca de tiempo) y producto:disponibilidad (carga útil: identificador del producto y su nueva disponibilidad), que mantiene la carta al día en todas las pantallas (RF-13).

Significado de los códigos de error: 400 entrada inválida · 401 token ausente, expirado o inválido · 403 rol sin permiso sobre la operación · 404 recurso inexistente · 409 transición de estado no permitida o restricción de integridad · 503 dependencia no disponible. El límite de intentos de inicio de sesión lo aplica el proveedor de identidad, fuera de este contrato.

2.5 Justificación del stack tecnológico

Las versiones se fijan de forma explícita y **nunca se usa la etiqueta `latest`**: con ella el servidor de producción puede levantar una versión distinta de la de desarrollo y romper el despliegue en plena entrega. Fijar la versión **reduce** esa diferencia entre entornos, aunque no la elimina por completo: etiquetas como `postgres:17-alpine` o `caddy:2-alpine` siguen recibiendo revisiones menores bajo el mismo nombre, de modo que las versiones efectivamente desplegadas se comprobarán en el servidor y se registrarán en el apartado 2.9. La primera mención de cada tecnología en el documento consigna su versión; las posteriores pueden omitirla.

Componente	Versión	Función en el sistema	Criterio que motivó su selección
Frontend — Flutter (Dart)	3.44.8 / Dart 3.12.2	Interfaz web de recepción y cocina; consume la API por HTTP y escucha los eventos en tiempo real	RNF-04 exige interfaz adaptable en Chrome y Edge, escritorio y tableta: un único código fuente cubre ambos anchos sin mantener dos interfaces. RNF-03 (estados cargando/vacio/error) se resuelve con el modelo declarativo de widgets. Es además la tecnología de interfaz trabajada en el diplomado, lo que hace verificable el dominio técnico exigido en la defensa
Backend — Node.js + Express	Node 24.15.0 LTS · npm 11.12.1	API REST, validación de token y rol, reglas de transición de estado y servidor del canal en tiempo real	RF-02, RF-05, RF-07 exigen que las reglas de negocio y la autorización se validen en el servidor. Alojar la API y el canal WebSocket en el mismo proceso permite emitir el evento en la misma operación que confirma el cambio (RF-03, RF-06). Se elige una versión LTS por soporte prolongado
Base de datos — PostgreSQL	17 (postgres:17-alpine)	Persistencia del dominio y única fuente de verdad	RF-02, RF-05, RF-07 y RNF-02 exigen integridad y trazabilidad: claves foráneas, restricciones CHECK y tipos enumerados garantizan en la propia base que el estado del pedido solo tome valores válidos. Las consultas parametrizadas del controlador pg previenen la inyección de SQL (RNF-02 , requisito mínimo 8). Se elige una versión mayor por debajo de la más reciente, con imágenes y controladores maduros
Tiempo real — Socket.IO (WebSockets)	4.x sobre Express	Propaga a recepción y cocina el alta de pedidos y los cambios de estado	RNF-01 fija la propagación en menos de 2 segundos y RF-03, RF-04 y RF-06 exigen ver el estado sin recargar. Frente a una plataforma de datos en tiempo real de terceros, esta opción mantiene una sola fuente de verdad (PostgreSQL) y una sola autenticación (Keycloak), y evita que el cliente acceda directamente a los datos. RNF-05 (degradación controlada) se implementa con los eventos de reconexión del propio canal
Autenticación — Keycloak	26.7 (quay.io/keycloak/keycloak:26.7)	Identidad, credenciales, roles y emisión de tokens firmados	RNF-02 y RF-01 exigen autenticación, contraseñas con hash, expiración del token y autorización por rol validada en el servidor. Resuelve con configuración —no con código propio— el almacenamiento seguro de contraseñas y el límite de intentos de inicio de sesión, cuya implementación artesanal sería la

Componente	Versión	Función en el sistema	Criterio que motivó su selección
			principal fuente de vulnerabilidades del proyecto. El servidor solo valida la firma mediante las claves publicadas (JWKS)
Contenedores — Docker / Docker Compose	Docker 20.10.1 · Compose v2.2.1	Entorno reproducible en desarrollo y en el servidor	El despliegue debe ser repetible y verificable (requisito mínimo 1). Las mismas imágenes y versiones que se prueban en desarrollo se levantan en el servidor, lo que elimina la clase de fallos “en mi máquina funciona”
Despliegue — Oracle Cloud Always Free + Caddy	Caddy 2 (caddy:2-alpine)	Servidor de producción, HTTPS y publicación del cliente y la API bajo el mismo origen	Requisito mínimo 1 exige dirección pública accesible y RNF-02 exige comunicación cifrada. Caddy obtiene y renueva el certificado TLS de forma automática, y servir cliente y API en el mismo origen elimina la configuración de CORS, una fuente frecuente de fallos en el primer despliegue. La capa gratuita permanente evita costo recurrente al negocio (factibilidad, apartado 1.4)
Control de versiones — Git + repositorio remoto	Git 2.x · GitHub	Historial del desarrollo y acceso del tribunal al código	Requisito mínimo 5 exige historial de avance progresivo y no una única carga final. Los commits se realizan manualmente por el autor, en fechas distintas y con mensajes que describen el cambio, lo que constituye a la vez la evidencia de la metodología del apartado 2.2

Tabla 13. Stack tecnológico y criterios de selección. Fuente: elaboración propia.

Bibliografía

Referencias citadas, según la norma ISO 690-2 para documentos electrónicos.

FETTE, Ian y MELNIKOV, Alexey. *RFC 6455: The WebSocket Protocol* [en línea]. Internet Engineering Task Force, diciembre de 2011 [consulta: 18 de septiembre de 2026]. Disponible en: <https://www.rfc-editor.org/rfc/rfc6455>

FIELDING, Roy Thomas. *Architectural styles and the design of network-based software architectures* [en línea]. Tesis doctoral. Irvine: University of California, 2000 [consulta: 18 de septiembre de 2026]. Disponible en: <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>

FUDO. *Software para restaurantes, bares y cafés* [en línea]. 2026 [consulta: 18 de septiembre de 2026]. Disponible en: <https://fu.do/>

LOYVERSE. *Sistema de pantalla de cocina para POS: Loyverse KDS* [en línea]. 2026 [consulta: 18 de septiembre de 2026]. Disponible en: <https://loyverse.com/es/kitchen-display-system>

OPEN SOURCE POINT OF SALE. *Open Source Point of Sale: repositorio oficial* [en línea]. Versión 3.4. 2026 [consulta: 18 de septiembre de 2026]. Disponible en: <https://github.com/opensourcepos/opensourcepos>

SAKIMURA, Nat, BRADLEY, John, JONES, Michael, DE MEDEIROS, Breno y MORTIMORE, Chuck. *OpenID Connect Core 1.0 incorporating errata set 2* [en línea]. OpenID Foundation, 15 de diciembre de 2023 [consulta: 18 de septiembre de 2026]. Disponible en: https://openid.net/specs/openid-connect-core-1_0.html

Verificación previa a la entrega

Los criterios coinciden con la rúbrica de E1.

	Verificación
☒	La tabla de antecedentes tiene de 3 a 5 soluciones, con la columna “qué deja sin resolver” completa y fuentes citables
☒	La descripción del problema identifica al menos tres puntos críticos con dato del contexto
☒	El objetivo general tiene verbo de acción, sistema, usuarios y ámbito delimitado
☒	Hay entre 5 y 6 objetivos específicos y todos tienen evidencia asignada en el Capítulo 2
☒	Todos los requisitos funcionales tienen al menos dos criterios de aceptación, uno de error
☒	Los Must have no superan el 60 % del backlog
☒	Los cinco requisitos no funcionales tienen métrica verificable
☒	Las capacidades fuera de alcance están declaradas y justificadas
☒	Los diagramas se hicieron con herramienta digital y son legibles en tamaño carta
☒	Todas las flechas del diagrama de arquitectura tienen protocolo y formato
☒	El modelo de datos tiene entre 4 y 8 entidades y su diccionario de datos
☒	La tabla del stack indica versión y criterio referido a un requisito por ID
☒	El repositorio existe, es accesible al docente y tiene README, .gitignore y .env.example
☒	El repositorio tiene al menos 5 commits con fechas distintas
☒	No hay ninguna credencial en el repositorio ni en su historial de Git
☒	Las capturas muestran datos de prueba, nunca datos personales reales
☒	Se eliminaron todos los recuadros grises de instrucciones de esta plantilla
☒	El documento se exportó también a PDF

Entrega: Moodle, tarea “E1 · Perfil de Proyecto”.